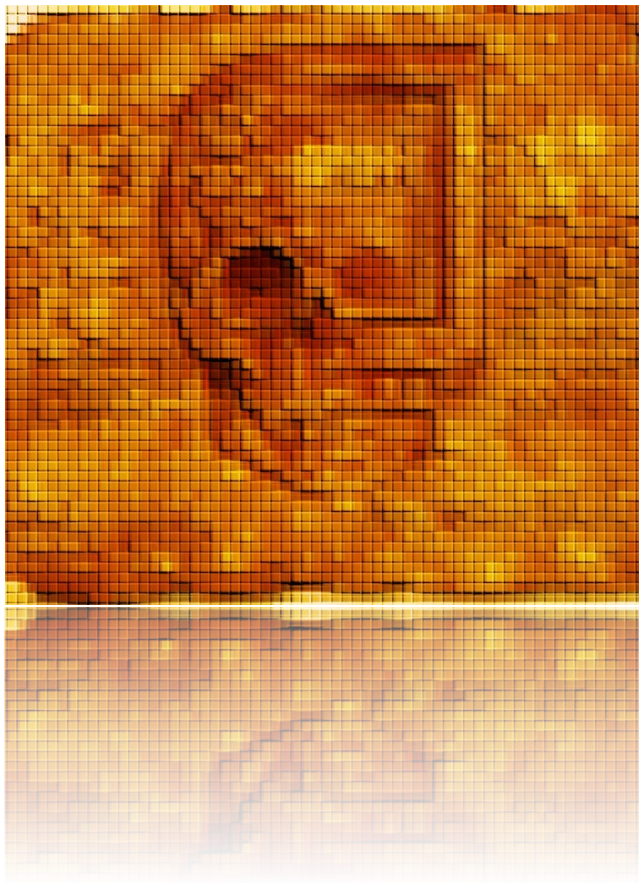
Computational Intelligence I: Neural Networks

Chapter 6: Radial Basis Function Networks



Pattern separability

⌘ Cover's theorem has that a set of linearly inseparable patterns, can become linearly separable in a higher dimensional space.

⌘ Let D={$\mathbf{x}_1,\ldots,\mathbf{x}_N$} be a set of d-dimensional patterns which belong to either of two classes C_1 or C_2.

◎ For each pattern $\mathbf{x}_i$ we define some real-valued functions $\varphi_j(\mathbf{x})$, for j=1,…b. This, in turn defines the multi-valued mapping:

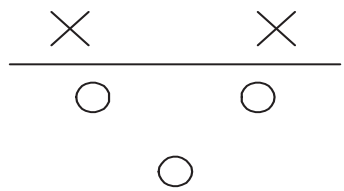
$$\varphi(\mathbf{x}) \equiv \left[\varphi_1(\mathbf{x}),\varphi_2(\mathbf{x}),\ldots,\varphi_b(\mathbf{x})\right]^T : \Re^d \rightarrow \Re^b$$

◎ A dichotomy (binary partition) of the samples in D, is φ-separable if there is a vector **w** such that:

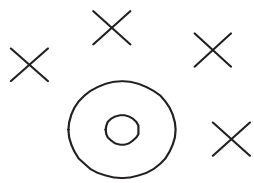
$$\exists \mathbf{w} \in \Re^b : \begin{cases} \mathbf{w}^T\varphi(\mathbf{x}) \geq 0 & \forall \mathbf{x} \in C_1 \\ \mathbf{w}^T\varphi(\mathbf{x}) < 0 & \forall \mathbf{x} \in C_2 \end{cases}$$

◎ The hyperplane $\mathbf{w}^T\varphi(\mathbf{x})=0$ partitions the b-dimensional φ-space to two sides. At the same time it partitions the d-dimensional input-space to two parts.

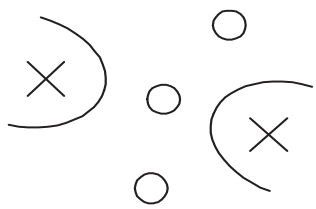
◎ Examples of such surfaces, include planes (a), spheres (b), quadrics (c), etc.



(a) Linearly seperable dichotomy



(b) Spherically seperable dichotomy



(c) Quadrically seperable dichotomy

An example of φ-separability:

⌘ This simple concept can be demonstrated with a simple example, such as the XOR problem.

◎ Without the need (for this example only) to increase the dimensionality d=2 of the input-space, we define b=2 functions:

$$\varphi_1(\mathbf{x}) \equiv \exp\left(-\|\mathbf{x}-\mathbf{c}_1\|_2^2\right), \quad \mathbf{c}_1 = (1,1)^T$$

$$\varphi_2(\mathbf{x}) \equiv \exp\left(-\|\mathbf{x}-\mathbf{c}_2\|_2^2\right), \quad \mathbf{c}_2 = (0,0)^T$$

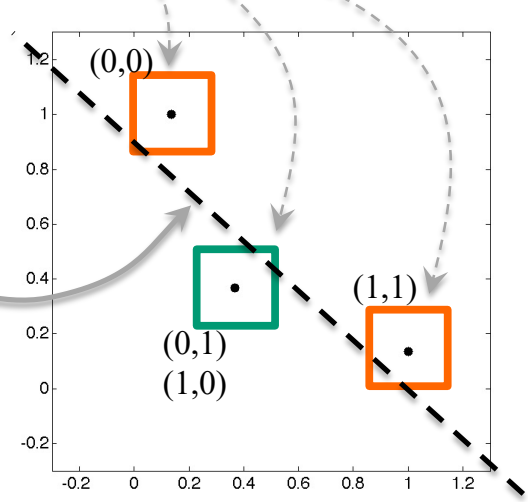
◎ Then, we can generate the following values:

Pattern x	$\varphi_1(x)$	$\varphi_2(x)$	Class
x= (1,1)	1	0.135	0
x= (0,1)	0.368	0.368	1
x= (0,0)	0.135	1	0
x= (1,0)	0.368	0.368	1

◎ This plane maps the 4 original patterns **x** in a way that are linearly separable.

◎ Thus, they can be classified with a simple algorithm such as a single layer perceptron, by finding some vector **w**, to define the decision boundary:

$$\boxed{\mathbf{w}^T\varphi(\mathbf{x}) = 0}$$



◎ Note: The decision boundary here happens to be linear for the particular choice of φ_1 and φ_2.

Interpolation

⌘ A very useful concept in many engineering areas, is that of interpolation, which given a set D of N d-dimensional points $\mathbf{x}_i$, and a corresponding set of associated values y_i, seeks to construct a function F, such that $F(\mathbf{x}_i)=y_i$ for all i=1,…,N; that is the surface defined by F to pass through all points $(\mathbf{x}_i,y_i)$.

⌘ One way for solving this problem is with the method of Radial Basis Functions (RBFs).

- This is based on defining a function:
  $$F(\mathbf{x}) \equiv \sum_{i=1}^{N} w_i \varphi\left(\left\|\mathbf{x}-\mathbf{x}_i\right\|\right)$$
- The N functions $\left\{\varphi\left(\left\|\mathbf{x}-\mathbf{x}_i\right\|\right)\right\}_{i=1}^{N}$ are arbitrary nonlinear RBFs.
- The norms $\|\cdot\|$ measure the distance of the argument $\mathbf{x}$ from the centres $\mathbf{x}_i$. For $\mathbf{x} = [x_1, \dots, x_d]^T$, $\|\mathbf{x}\| = \sqrt{x_1^2 + \cdots + x_d^2}$
- Combing the above RBF expression with the interpolation condition, we have:

$$\underbrace{\begin{bmatrix} \varphi_{11} & \varphi_{12} & \cdots & \varphi_{1N} \\ \varphi_{21} & \varphi_{22} & \cdots & \varphi_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \varphi_{N1} & \varphi_{N2} & \cdots & \varphi_{NN} \end{bmatrix}}_{\boldsymbol{\Phi}} \underbrace{\begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix}}_{\mathbf{w}} = \underbrace{\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}}_{\mathbf{y}}, \quad \text{where} \begin{cases} \varphi_{ji} \equiv \varphi\left(\left\|\mathbf{x}_j - \mathbf{x}_i\right\|\right) \\ \text{for } j,i = 1,\dots,N \end{cases}$$

e.g., $F(\mathbf{x}_2) = y_2$

⌘ Continued…

- The previous equation can be more compactly expressed and solved, in terms of the $N \times N$ interpolation matrix Φ, the $N \times 1$ weight vector **w** and the $N \times 1$ desired response **y**, as:

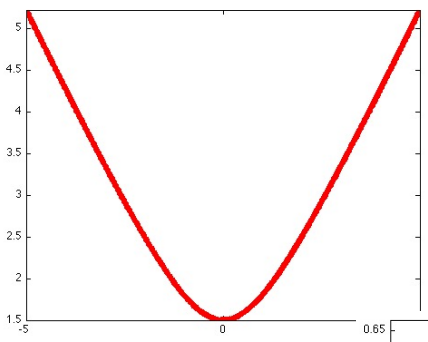
$$\mathbf{\Phi w} = \mathbf{y} \Rightarrow \mathbf{w} = \mathbf{\Phi}^{-1}\mathbf{y}$$

- The above assumes that Φ is nonsingular so that its inverse exists. This is true for a large class of RBFs (assuming the N points are different) such as:

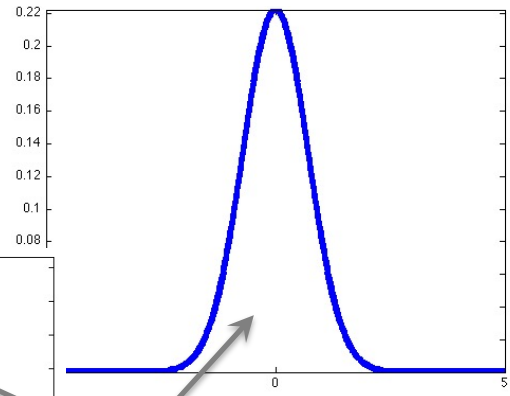
- <u>Multiquadrics</u>: $\boxed{\varphi(r) = \sqrt{r^2 + c^2}, \quad c > 0, r \in \Re}$

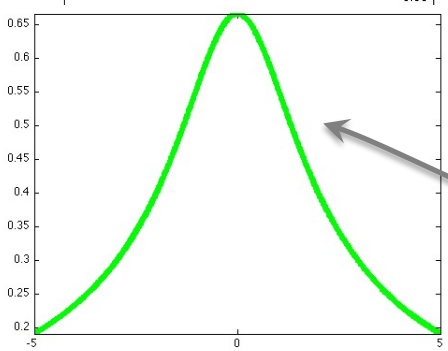
- <u>Inverse multiquadrics</u>: $\boxed{\varphi(r) = \dfrac{1}{\sqrt{r^2 + c^2}}, \quad c > 0, r \in \Re}$

- <u>Gaussian functions</u>: $\boxed{\varphi(r) = \exp\left(-\dfrac{r^2}{2\sigma^2}\right), \quad \sigma > 0, r \in \Re}$



All plots have:
c= σ= 1.5





These are localised functions, since $\varphi(r) \rightarrow 0$, when $r \rightarrow \infty$.

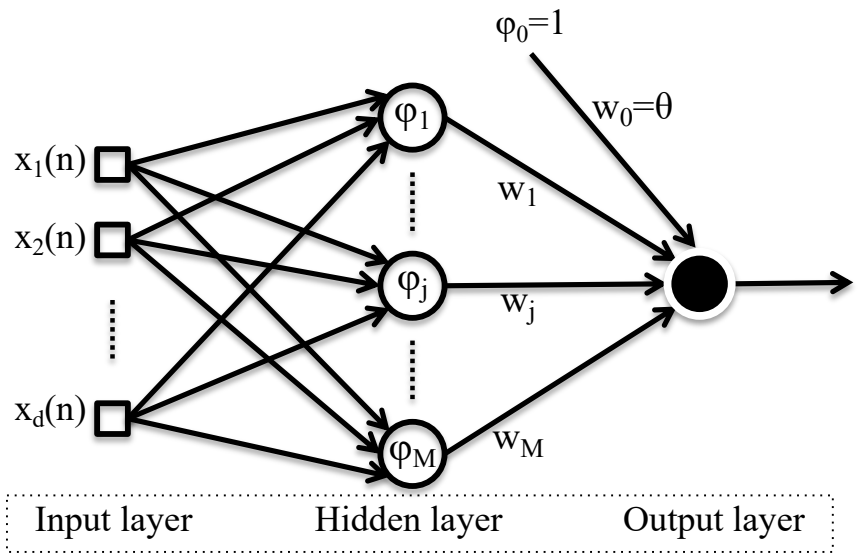
RBF Neural Networks

⌘ An RBF neural network can learn from a set of N data patterns, as follows:

- The overall network function can be represented as:

$$F(\mathbf{x}) = \sum_{i=1}^{M} w_i \varphi_i(\mathbf{x}), \quad \varphi_i(\mathbf{x}) \equiv \exp\left(-\frac{\|\mathbf{x}-\mathbf{c}_i\|_2^2}{2\sigma_i^2}\right)$$

- The Gaussians are a frequent choice for such networks but not unique.
- We can use a set of M≤N basis functions φ_i but <u>typically</u> M<<N, as it is very computationally inefficient (to invert and store an N × N matrix) and usually an exaggeration in terms of network power and complexity to use as many functions as the number of samples N.
- Each φ_i is <u>centred</u> at $\mathbf{c}_i$. Using M=N and $\mathbf{c}_i=\mathbf{x}_i$, we get the interpolation model previously discussed.
- The parameter σ_i (or full covariance Σ for the Gaussian) determines the <u>receptive field</u> of each RBF. This is the subset of the domain of the input **x** which receives the environment stimuli.



⌘ Continued…

- In order to train this model, we define its overall network error in terms of its weights (for the moment assume centres $\mathbf{c}_i$ and widths σ_i are given) which depends on a data **fitting term**, plus a **regularisation term** (similar to the MLP case for controlling complexity and avoiding overfitting):

$$E(\mathbf{w}) = \sum_{i=1}^{N}\left[F(\mathbf{x}_i) - y_i\right]^2 + \lambda\left\|\mathbf{D}F\right\|_2^2 = \sum_{i=1}^{N}\left[\underbrace{\sum_{j=1}^{M} w_j\varphi_j(\mathbf{x}_i)}_{F(\mathbf{x}_i)} - y_i\right]^2 + \lambda\left\|\mathbf{D}F\right\|^2$$

- As for the interpolation case, we can express the fitting error in the more compact notation:

$$\left\|\underbrace{\begin{bmatrix}\varphi_1(x_1) & \cdots & \varphi_M(x_1)\\ \vdots & \ddots & \vdots\\ \varphi_1(x_N) & \cdots & \varphi_M(x_N)\end{bmatrix}}_{\Phi\in\Re^{N\times M}}\underbrace{\begin{bmatrix}w_1\\ \vdots\\ w_M\end{bmatrix}}_{\mathbf{w}\in\Re^{M\times 1}} - \underbrace{\begin{bmatrix}y_1\\ \vdots\\ y_N\end{bmatrix}}_{\mathbf{y}\in\Re^{N\times 1}}\right\|_2^2 \Leftrightarrow \left\|\Phi\mathbf{w} - \mathbf{y}\right\|_2^2$$

- The regularisation term is employing a linear differential operator **D**, (e.g., $\|\mathbf{D}F\| = \|\mathbf{w}\| = \mathbf{w}^T\mathbf{w}$) which usually incorporates some prior knowledge about the solution. λ is the user-defined regularisation parameter. When λ➔0, no smoothness in sought, while λ➔∞ implies the data patterns are unreliable.

- The above minimisation problem is usually solved using the least square solution based on the pseudoinverse:

$$\mathbf{w} = \Phi^+\mathbf{y}, \quad \text{where } \Phi^+ \equiv \lim_{\lambda\to 0}\left(\Phi^T\Phi + \lambda I_{M\times M}\right)^{-1}\Phi^T$$

- This pseudoinverse is efficiently solved using Singular Value Decomposition (SVD) techniques.

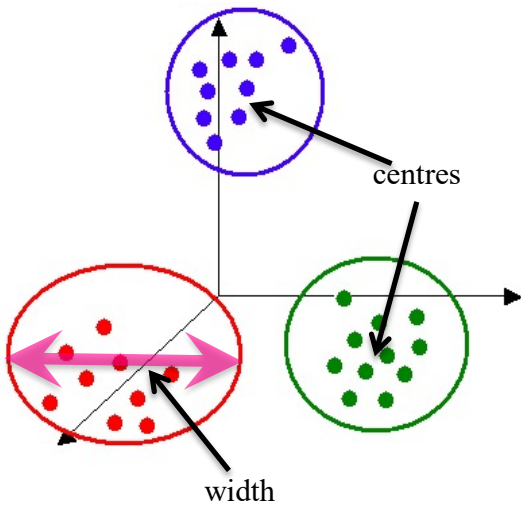
RBF Learning strategies

⌘ In the RBF discussion so far, we ignored the definition of the activation functions φ_i and concentrated on the weights of the output layer only. Firstly, activations (centres c_i and widths σ_i) were set, and <u>then</u> weights were optimised.

⌘ Various strategies exist to <u>also</u> set centres and basis widths:

- **Random selection of fixed centres**: If the N training samples are uniformly distributed, a random subset of M<<N of them can serve as the centres. Then, using the maximum distance d_{max} between the chosen centres we set: $\sigma = d_{max} / \sqrt{2M}$
- This ensures that the RBFs are not too peaky or flat. Alternatively, we can set larger or smaller σ_i in areas of low or higher data density.
- Then, weights are set using the least squares fit we described earlier using the pseudoinverse.

- **Self-organised centre selection**: The previous method requires a lot of data samples. Therefore, this method uses a clustering algorithm to group the data points into distinct groups.
- Then, the cluster locations and spreads are used to compute centres and widths of the RBFs,
- Finally, weights can be calculated with the least squares procedure.



⌘ Continued…

- **Supervised centre selection**: This method optimises all three types of parameters (centres, widths and weights) simultaneously, by minimising the (unregularised here) error:

$$E\left(\{w_i, \mathbf{c}_i, \sigma_i\}\right) = \frac{1}{2}\sum_{j=1}^{N} e_j^2 = \frac{1}{2}\sum_{j=1}^{N}\left[\sum_{i=1}^{M} w_i \exp\left(\frac{-\left\|\mathbf{x}_j - \mathbf{c}_i\right\|_2^2}{2\sigma_i^2}\right) - y_j\right]^2$$

- The three different parameter types can be optimised by stochastic gradient using three different learning rates η_1, η_2 & η_3.

- Such an algorithm could be based on the following scheme (*we assume here that there is no full covariance for the Gaussian but just one parameter σ_i for each function*):

- <u>Adapting linear weights in the output layer</u>:

$$\boxed{w_i(n+1)} = w_i(n) - \eta_1 \frac{\partial E(n)}{\partial w_i(n)}, \; i = 1,\ldots,M$$

$$\text{where } \boxed{\frac{\partial E(n)}{\partial w_i(n)}} = \sum_{j=1}^{N} e_j \exp\left(\frac{-\left\|\mathbf{x}_j - \mathbf{c}_i\right\|_2^2}{2\sigma_i^2}\right)$$

Dropping (n) in RHS

- <u>Adapting centre positions</u>:

$$\boxed{c_{ik}(n+1)} = c_{ik}(n) - \eta_2 \frac{\partial E(n)}{\partial c_{ik}(n)}, \; i = 1,\ldots,M, k = 1,\ldots,d$$

$$\text{where } \boxed{\frac{\partial E(n)}{\partial c_{ik}(n)}} = 2w_i \sum_{j=1}^{N} e_j \frac{x_{jk} - c_{ik}}{2\sigma_i^2} \exp\left(\frac{-\left\|\mathbf{x}_j - \mathbf{c}_i\right\|_2^2}{2\sigma_i^2}\right)$$

Dropping (n) in RHS

- Adapting basis function widths:

$$\boxed{\sigma_i(n+1)} = \sigma_i(n) - \eta_3 \frac{\partial E(n)}{\sigma_i(n)}, \; i = 1,\ldots,M$$

$$\text{where } \boxed{\frac{\partial E(n)}{\sigma_i(n)}} = -w_i \sum_{j=1}^{N} e_j \frac{\left\|\mathbf{x}_j - \mathbf{c}_i\right\|_2^2}{\sigma_i^3} \exp\left(\frac{-\left\|\mathbf{x}_j - \mathbf{c}_i\right\|_2^2}{2\sigma_i^2}\right)$$

Dropping (n) in RHS

RBFs vs MLPs

⌘ What are the typical similarities and differences between RBF and MLP networks?

- Both are ANNs for nonlinear mapping.
- Both are Universal Approximators.
- RBFs have single hidden layers, while MLPs may have more than one hidden layers.
- The neurons of an MLP in a hidden or output layer often share a common modelling task. The neurons of an RBF network in the hidden and output layers serve completely different tasks.
- The RBF hidden nodes are nonlinear, while the output nodes linear in weights. Both hidden and output MLP nodes are nonlinear (apart from when an MLP is solving a regression problem!).
- The RBF hidden activation is based on a function of the distance of the input vector and the RBF centre (defining the receptive field). The activations of hidden MLP nodes are based on the inner product of the input vector or previous layer input and the synaptic weight vectors for that neuron.
- MLPs construct global approximations to nonlinear input-to-output mapping. RBFs construct localised approximations to such nonlinear mappings. This could mean that MLPs can do the same task with fewer parameters.

<END of Chapter 6>